

Lab Assignment 16.1

Name : Kathyayani Choudhari(46)

Roll No : 2303A53001

Task 1 – Student Information System Schema

Task: Design a database schema for a Student Information System and generate queries using AI.

Instructions: • Tables: Students, Courses, Enrollments.

- Define primary keys, foreign keys, and relationships.
- Generate queries:
 - o Insert a new student record.
 - o Fetch all courses enrolled by a student.
 - o Count number of students in each course.

Expected Output:

- SQL CREATE TABLE statements and queries for student–course relationships.

Prompt :

Design a Student Information System database with tables: Students, Courses, and Enrollments.

Define primary keys and foreign keys.

Also generate SQL queries to:

1. Insert a new student
2. Fetch all courses enrolled by a student

3. Count number of students in each course

Code :

-- Students Table

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100)  
);
```

-- Courses Table

```
CREATE TABLE Courses (  
    course_id INT PRIMARY KEY,  
    course_name VARCHAR(100)  
);
```

-- Enrollments Table (Relationship Table)

```
CREATE TABLE Enrollments (  
    enrollment_id INT PRIMARY KEY,  
    student_id INT,  
    course_id INT,  
    FOREIGN KEY (student_id) REFERENCES Students(student_id),  
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
);
```

-- Insert a new student

```
INSERT INTO Students (student_id, name, email)
VALUES (1, 'Kathyayani', 'kathyayani@email.com');
```

-- Fetch all courses enrolled by a student

```
SELECT c.course_name
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
WHERE e.student_id = 1;
```

-- Count number of students in each course

```
SELECT c.course_name, COUNT(e.student_id) AS total_students
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
```

Output :

Student Table

student_id	name	email
1	Kathyayani	kathyayanichludhari@gmail.com

Courses Enrolled

course_name
DBMS
OS

Student Count per Course

course_name	Total_students
DBMS	10
OS	8

Explanation :

Schema:

Students & Courses are main tables; Enrollments links them.
It creates a many-to-many relationship.

Insert Query:

Adds a new student to the Students table.
Each student has a unique ID.

Fetch Courses:

JOIN gets courses for a specific student.
Uses student_id to filter.

Count Students:

Counts students in each course using GROUP BY.
Shows all courses even if empty.

Task 2 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions: • Tables: Doctors, Patients, Appointments.

- Use AI to define constraints (unique IDs, valid dates).
- Generate queries:
 - o List all appointments for a specific doctor.
 - o Retrieve patient history by patient ID.

o Count total patients treated by each doctor.

Expected Output:

- Normalized schema and SQL queries with joins.

Prompt :

Design a Hospital Management System database with tables: Doctors, Patients, and Appointments.

Define primary keys, foreign keys, and constraints like unique IDs and valid appointment dates.

Generate SQL queries to:

1. List all appointments for a specific doctor
2. Retrieve patient history by patient ID
3. Count total patients treated by each doctor

Code:

-- Doctors Table

```
CREATE TABLE Doctors (  
    doctor_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    specialization VARCHAR(100)  
);
```

-- Patients Table

```
CREATE TABLE Patients (  
    patient_id INT PRIMARY KEY,
```

```
name VARCHAR(100),  
age INT CHECK (age > 0)  
);
```

-- Appointments Table

```
CREATE TABLE Appointments (  
    appointment_id INT PRIMARY KEY,  
    doctor_id INT,  
    patient_id INT,  
    appointment_date DATE NOT NULL,  
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),  
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)  
);
```

-- List all appointments for a specific doctor

```
SELECT a.appointment_id, p.name AS patient_name, a.appointment_date  
FROM Appointments a  
JOIN Patients p ON a.patient_id = p.patient_id  
WHERE a.doctor_id = 1;
```

-- Retrieve patient history by patient ID

```
SELECT a.appointment_id, d.name AS doctor_name, a.appointment_date  
FROM Appointments a  
JOIN Doctors d ON a.doctor_id = d.doctor_id  
WHERE a.patient_id = 1;
```

-- Count total patients treated by each doctor

```
SELECT d.name AS doctor_name, COUNT(DISTINCT a.patient_id) AS  
total_patients  
FROM Doctors d  
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id  
GROUP BY d.name;
```

Output :

Appointments for Doctor

appointment_id	patient_name	appointment_date
101	Ravi	2026-03-20

Patient History

appointment_id	doctor_name	appointment_date
101	Dr.Sharma	2026-03-20

Patients per Doctor

doctor_name	Total_patients
Dr.Sharma	5

Explanation :

Schema:

Doctors & Patients are main tables; Appointments links them.
Constraints ensure valid age and non-null dates.

Doctor Appointments:

JOIN gets patient details for a doctor.
Filters using doctor_id.

Patient History:

Shows all visits of a patient with doctors.

Uses patient_id filter.

Count Patients:

Counts unique patients per doctor.

LEFT JOIN includes doctors with no patients.

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions: • Tables: Books, Members, Loans.

- Use AI to suggest indexing strategy for faster queries.
- Generate queries:
 - o Retrieve all books currently issued.
 - o Find overdue books (loan date > 30 days).
 - o Count number of books loaned by each member.

Expected Output:

- Schema + SQL queries demonstrating joins and conditions.

Prompt :

Design a Library Management System database with tables: Books, Members, and Loans.

Define primary keys, foreign keys, and suggest indexing for faster queries.

Generate SQL queries to:

1. Retrieve all books currently issued
2. Find overdue books (loan date > 30 days)

3. Count number of books loaned by each member

Code :

-- Books Table

```
CREATE TABLE Books (  
    book_id INT PRIMARY KEY,  
    title VARCHAR(100),  
    author VARCHAR(100)  
);
```

-- Members Table

```
CREATE TABLE Members (  
    member_id INT PRIMARY KEY,  
    name VARCHAR(100)  
);
```

-- Loans Table

```
CREATE TABLE Loans (  
    loan_id INT PRIMARY KEY,  
    book_id INT,  
    member_id INT,  
    loan_date DATE,  
    return_date DATE,  
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
```

```
FOREIGN KEY (member_id) REFERENCES Members(member_id)
);
```

-- Indexing for faster queries

```
CREATE INDEX idx_book ON Loans(book_id);
CREATE INDEX idx_member ON Loans(member_id);
```

-- Retrieve all books currently issued (not returned)

```
SELECT b.title, m.name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;
```

-- Find overdue books (> 30 days)

```
SELECT b.title, m.name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL
AND l.loan_date < CURRENT_DATE - INTERVAL 30 DAY;
```

-- Count books loaned by each member

```
SELECT m.name, COUNT(l.book_id) AS total_books
FROM Members m
```

LEFT JOIN Loans l ON m.member_id = l.member_id

GROUP BY m.name;

Output :

Currently Issued Books

title	name	loan_date
DBMS Basics	Anu	2026-03-01

Overdue Books

title	name	loan_date
OS Concepts	Ravi	2026-02-10

Books per Member

name	total_books
Anu	3
Ravi	2

Explanation :

Books & Members are main tables; Loans connects them.

Indexes on foreign keys speed up search.

Issued Books:

Finds books not yet returned.

Checks return_date IS NULL.

Overdue Books:

Filters books older than 30 days.

Uses date condition.

Count Books:

Counts loans per member.

LEFT JOIN includes members with zero loans.

Task 4 – Real-Time Application: Online Shopping Database

Scenario: Design a database for an E-commerce platform.

Requirements: • Tables: Users, Products, Orders, OrderDetails.

- Generate queries to:
 - o Retrieve all orders by a user.
 - o Find the most purchased product.
 - o Calculate total revenue in a given month.
- AI should also suggest normalization improvements and query optimization.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

Prompt :

Design an E-commerce database with tables: Users, Products, Orders, and OrderDetails.

Define primary keys, foreign keys, and relationships.

Generate SQL queries to:

1. Retrieve all orders by a user
2. Find the most purchased product
3. Calculate total revenue in a given month

Also suggest normalization improvements and query optimization techniques.

Code:

-- Users Table

```
CREATE TABLE Users (  
    user_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100) UNIQUE  
);
```

-- Products Table

```
CREATE TABLE Products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100),  
    price DECIMAL(10,2)  
);
```

-- Orders Table

```
CREATE TABLE Orders (  
    order_id INT PRIMARY KEY,  
    user_id INT,  
    order_date DATE,  
    FOREIGN KEY (user_id) REFERENCES Users(user_id)  
);
```

-- OrderDetails Table

```
CREATE TABLE OrderDetails (  
    order_detail_id INT PRIMARY KEY,  
    order_id INT,  
    product_id INT,  
    quantity INT CHECK (quantity > 0),  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),  
    FOREIGN KEY (product_id) REFERENCES Products(product_id)  
);
```

-- Indexing for optimization

```
CREATE INDEX idx_user ON Orders(user_id);  
CREATE INDEX idx_product ON OrderDetails(product_id);
```

-- 1. Retrieve all orders by a user

```
SELECT o.order_id, o.order_date  
FROM Orders o  
WHERE o.user_id = 1;
```

-- 2. Most purchased product

```
SELECT p.product_name, SUM(od.quantity) AS total_sold  
FROM OrderDetails od  
JOIN Products p ON od.product_id = p.product_id  
GROUP BY p.product_name  
ORDER BY total_sold DESC  
LIMIT 1;
```

-- 3. Total revenue in a given month

```
SELECT SUM(p.price * od.quantity) AS total_revenue
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
JOIN Orders o ON od.order_id = o.order_id
WHERE MONTH(o.order_date) = 3 AND YEAR(o.order_date) = 2026;
```

Output :

Orders by User

order_id	order_id
101	2026-03-10
102	2026-03-15

Most Purchased Product

product_name	total_sold
Laptop	50

Total Revenue

total_revenue
250000

Explanation :

Schema:

Users, Products, Orders are main tables; OrderDetails handles many-to-many.

Foreign keys maintain relationships.

User Orders:

Fetches orders using user_id.

Simple filtering.

Most Purchased:

Groups products and sums quantity.

Sorts to get top product.

Revenue:

Multiplies price × quantity.

Filters by month & year.

Task 5 – University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

Instructions: • Tables: Students, Subjects, Exams, Results.

- Define primary keys, foreign keys, and referential integrity constraints.

- Generate queries:

- o Insert examination results for a student.

- o Retrieve subject-wise marks for a student.

- o Calculate average marks for each subject.

Expected Output:

- Normalized schema and SQL queries involving aggregation and joins.

Prompt :

Design a University Examination System database with tables: Students, Subjects, Exams, and Results.

Define primary keys, foreign keys, and referential integrity constraints.

Generate SQL queries to:

1. Insert examination results for a student
2. Retrieve subject-wise marks for a student
3. Calculate average marks for each subject

Code :

-- Students Table

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(100)  
);
```

-- Subjects Table

```
CREATE TABLE Subjects (  
    subject_id INT PRIMARY KEY,  
    subject_name VARCHAR(100)  
);
```

-- Exams Table

```
CREATE TABLE Exams (  
    exam_id INT PRIMARY KEY,  
    subject_id INT,  
    exam_date DATE,  
    FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id)  
);
```

-- Results Table

```
CREATE TABLE Results (  
    result_id INT PRIMARY KEY,  
    student_id INT,  
    exam_id INT,  
    marks INT CHECK (marks >= 0),  
    FOREIGN KEY (student_id) REFERENCES Students(student_id),  
    FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)  
);
```

-- Insert examination result

```
INSERT INTO Results (result_id, student_id, exam_id, marks)  
VALUES (1, 1, 101, 85);
```

-- Retrieve subject-wise marks for a student

```
SELECT s.subject_name, r.marks  
FROM Results r  
JOIN Exams e ON r.exam_id = e.exam_id  
JOIN Subjects s ON e.subject_id = s.subject_id  
WHERE r.student_id = 1;
```

-- Calculate average marks for each subject

```
SELECT s.subject_name, AVG(r.marks) AS avg_marks  
FROM Results r
```

JOIN Exams e ON r.exam_id = e.exam_id

JOIN Subjects s ON e.subject_id = s.subject_id

GROUP BY s.subject_name;

Output :

Inserted Result

result_id	student_id	exam_id	marks
1	1	1	1

Subject-wise Marks

subject_name	marks
DBMS	85
OS	78

Average Marks

subject_name	avg_marks
DBMS	80
OS	75

Explanation :

Schema:

Students & Subjects are main tables; Exams and Results link them.

Foreign keys ensure valid relationships.

Insert Result:

Adds marks for a student in an exam.

CHECK ensures valid marks.

Subject-wise Marks:

JOIN connects results to subjects.

Filters by student_id.

Average Marks:

Calculates average using AVG().

GROUP BY subject for aggregation.